TD - Passage du style procédural au style orienté objet

Voici un programme écrit en langage Python dans un style de programmation procédural.

Version procédurale

```
def publish_file(file_src_path, file_dst_path, export_type):
    # Zone 3
    publish_file_path = file_dst_path
    match export_type:
        case None:
            return None
        case "copy":
            shutil.copyfile(file_src_path, publish_file_path)
        case "pdf-slideshow":
            publish_file_path = file_dst_path.with_suffix(".pdf")
            subprocess.run(["npx", "marp-cli", "--pdf", file_src_path,
↪   "-o", publish_file_path])
        case "pandoc":
        publish_file_path = file_dst_path.with_suffix(".pdf")
            subprocess.run(["pandoc", "-o", publish_file_path,
↪   file_src_path])
        case _:
            raise RuntimeError(f"Unknown export type : {export_type}")
    return publish_file_path

def main():
    # Zone 1
    parser = argparse.ArgumentParser()
    parser.add_argument("-o", "--output", required=True)
    args = parser.parse_args()
    args.output = Path(args.output)

    if not args.output.is_dir():
        raise RuntimeError("args.output need to be a directory.")

    tree = {}
    root_dir = Path("../src")
    file_formats = ".md"

    attachments_dir = args.output / "attachments"
```

```
    if attachments_dir.exists():
        shutil.rmtree(attachments_dir)
    # Zone 2
    for d in os.listdir(root_dir):
        tree[d] = {}
        for sd in os.listdir(root_dir / d):
            final_src_path = root_dir / d / sd
            final_dst_path = attachments_dir / d / sd
            os.makedirs(final_dst_path)
            tree[d][sd] = []
            for f in os.listdir(final_src_path):
                if Path.is_file(final_src_path / f) and
                ↪  f.endswith(file_formats):
                    p_file = publish_file(
                        final_src_path / f,
                        final_dst_path / f,
                        export_type=extract_export_type_from_name(f),
                    )
                    if p_file:
                        tree[d][sd].append(p_file.name)

    # Zone 4
    with open(args.output / "_data" / "attachments.json", "w") as
    ↪  output_file:
        output_file.write(json.dumps(tree, ensure_ascii=False))

if __name__ == "__main__":
    main()
```

1 - Expliquez le rôle de chaque zone identifiée par un commentaire # Zone x

2 - Quelle modification est nécessaire si l'on souhaite ajouter un format de fichier en export ?

Version orientée objet

3 - Ecrivez une version orientée objet de ce programme

```
def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("-o", "--output", required=True)
    args = parser.parse_args()
    args.output = Path(args.output)

    if not args.output.is_dir():
        raise RuntimeError("args.output need to be a directory.")

    tree = {}
    root_dir = Path("../src")
    file_formats = ".md"

    attachments_dir = args.output / "attachments"
```

```
    if attachments_dir.exists():
        shutil.rmtree(attachments_dir)

    for d in os.listdir(root_dir):
        tree[d] = {}
        for sd in os.listdir(root_dir / d):
            final_src_path = root_dir / d / sd
            final_dst_path = attachments_dir / d / sd
            os.makedirs(final_dst_path)
            tree[d][sd] = []
            for f in os.listdir(final_src_path):
                if Path.is_file(final_src_path / f) and
                ↪  f.endswith(file_formats):

                    if p_file:
                        tree[d][sd].append(p_file.name)

    with open(args.output / "_data" / "attachments.json", "w") as
    ↪  output_file:
        output_file.write(json.dumps(tree, ensure_ascii=False))

if __name__ == "__main__":
    main()
```